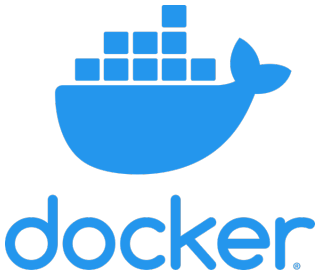
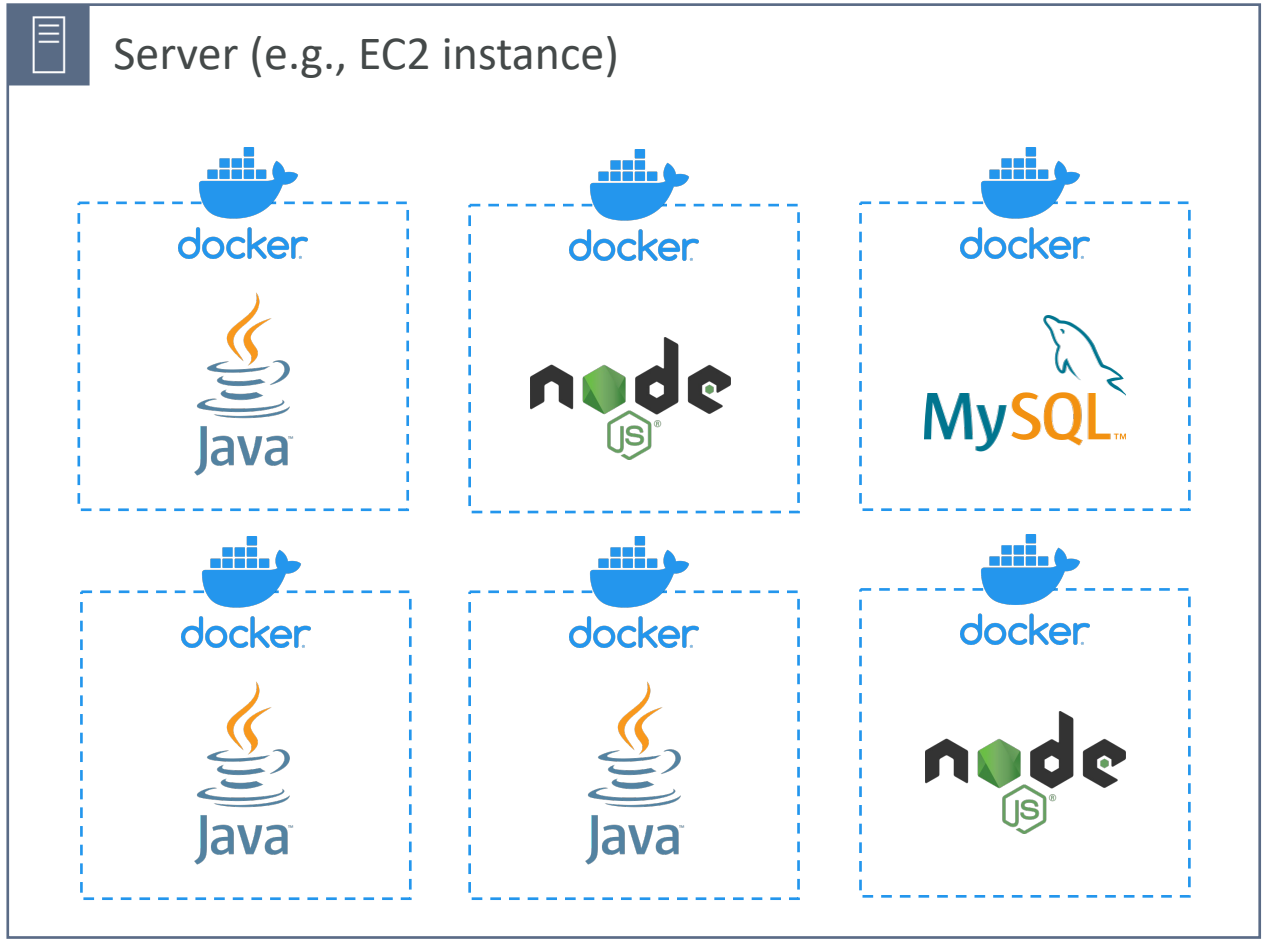
Containers on AWS

What is Docker?



- Docker is a software development platform to deploy apps
- Apps are packaged in **containers** that can be run on any OS
- Apps run the same, regardless of where they're run
  - Any machine
  - No compatibility issues
  - Predictable behavior
  - Less work
  - Easier to maintain and deploy
  - Works with any language, any OS, any technology
- Use cases: microservices architecture, lift-and-shift apps from on-premises to the AWS cloud, …

Docker on an OS



Where are Docker images stored?

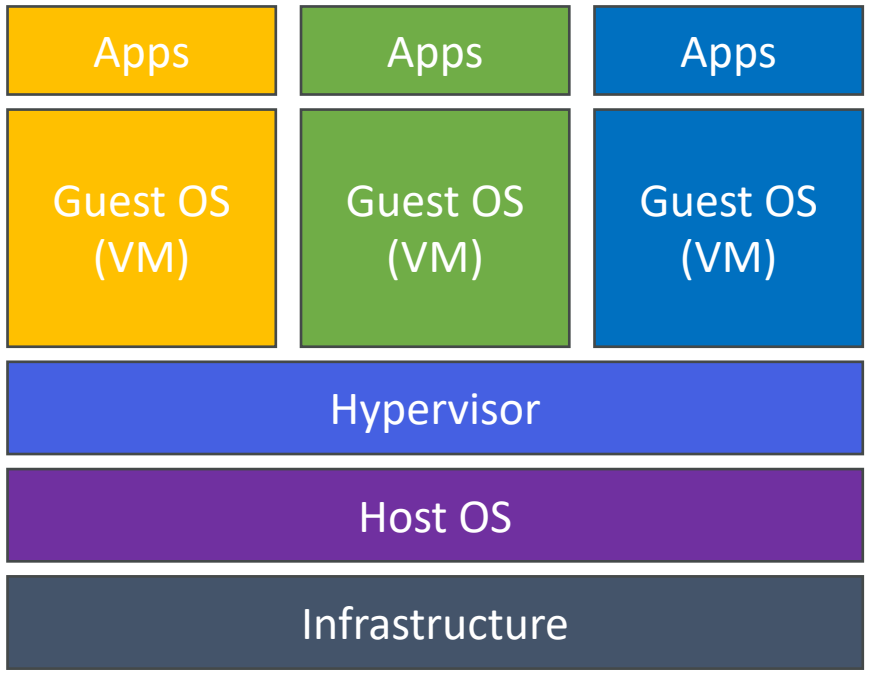
- Docker images are stored in Docker Repositories

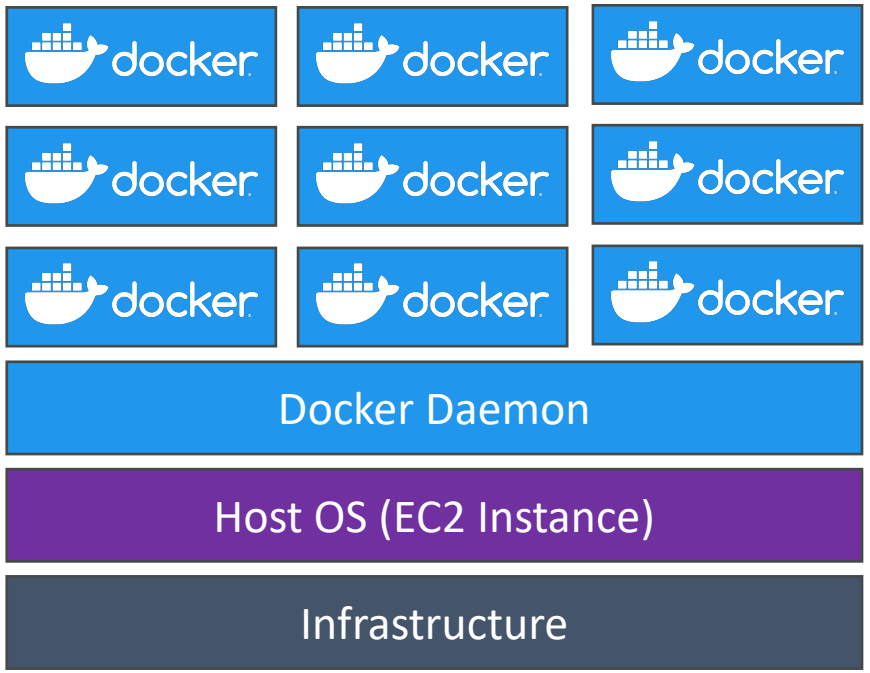
- Docker Hub (https://hub.docker.com)
  - **Public** repository
  - Find base images for many technologies or OS (e.g., Ubuntu, MySQL, …)

- Amazon ECR (Amazon Elastic Container Registry)
  - **Private** repository
  - **Public** repository (**Amazon ECR Public Gallery** https://gallery.ecr.aws)

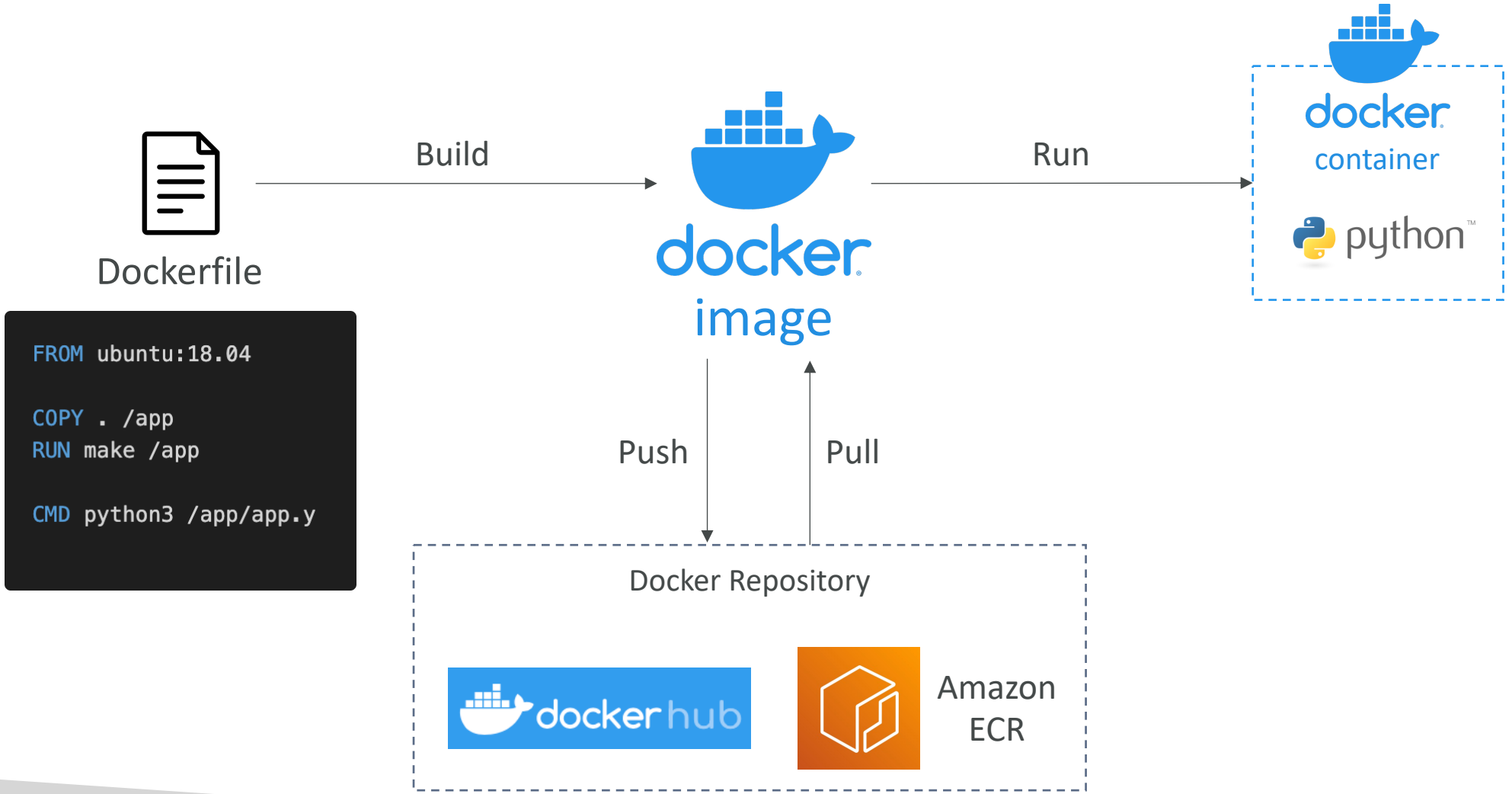
Docker vs. Virtual Machines

- Docker is "sort of" a virtualization technology, but not exactly
- Resources are shared with the host => many containers on one server





Getting Started with Docker



Docker Containers Management on AWS

- Amazon Elastic Container Service (Amazon ECS)
  - Amazon's own container platform
- Amazon Elastic Kubernetes Service (Amazon EKS)
  - Amazon's managed Kubernetes (open source)
- AWS Fargate
  - Amazon's own Serverless container platform
  - Works with ECS and with EKS
- Amazon ECR:
  - Store container images

Amazon ECS

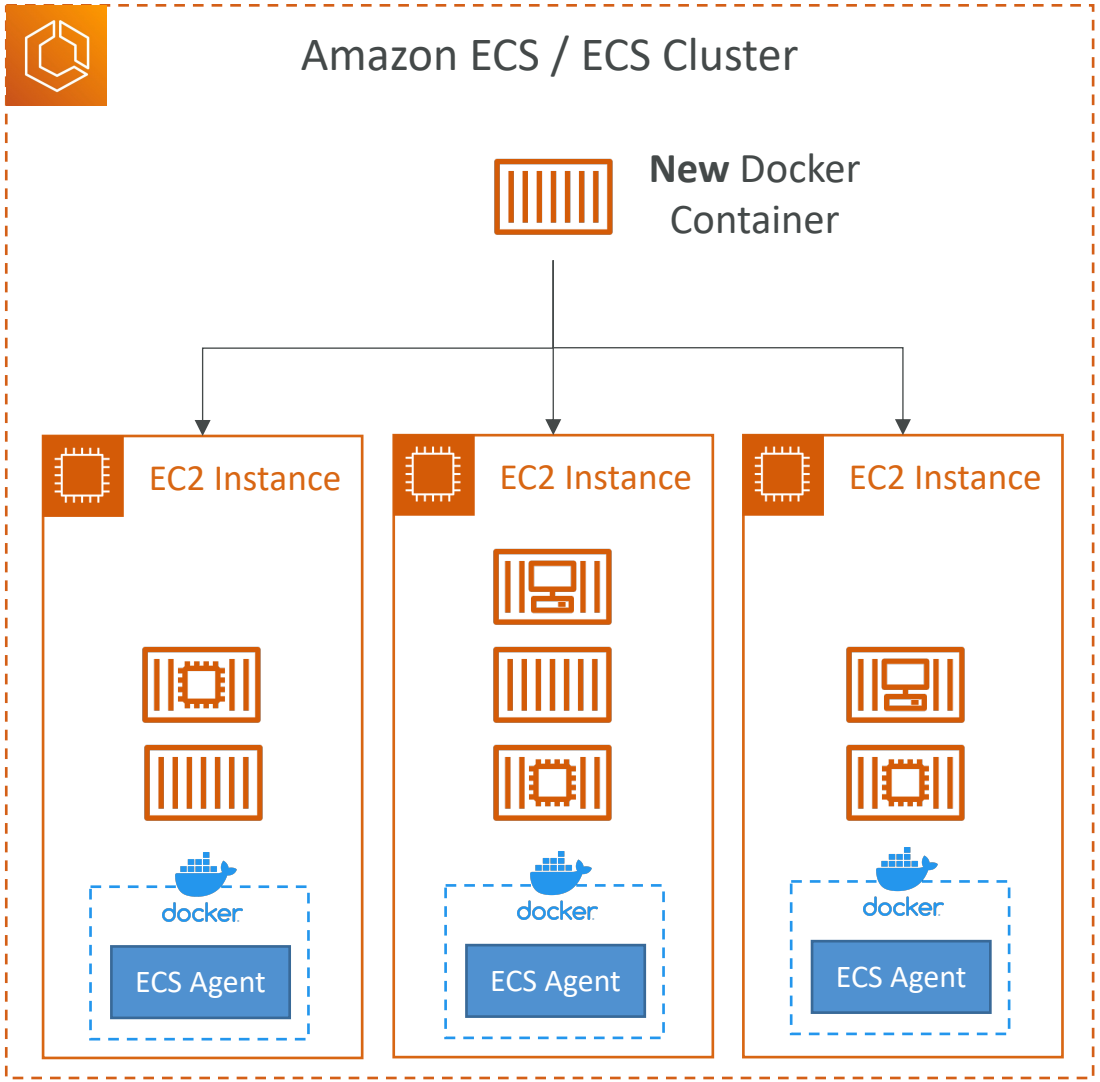
Amazon EKS

AWS Fargate

Amazon ECR

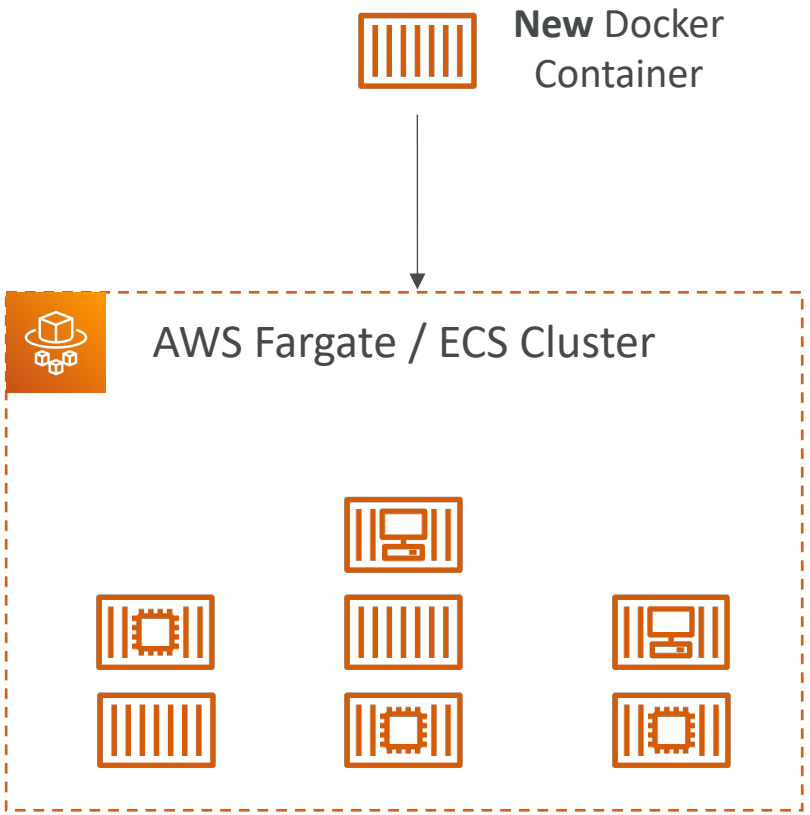
Amazon ECS - EC2 Launch Type

- ECS = Elastic Container Service
- Launch Docker containers on AWS = Launch **ECS Tasks** on ECS Clusters
- **EC2 Launch Type: you must provision & maintain the infrastructure (the EC2 instances)**
- Each EC2 Instance must run the ECS Agent to register in the ECS Cluster
- AWS takes care of starting / stopping containers



Amazon ECS – Fargate Launch Type

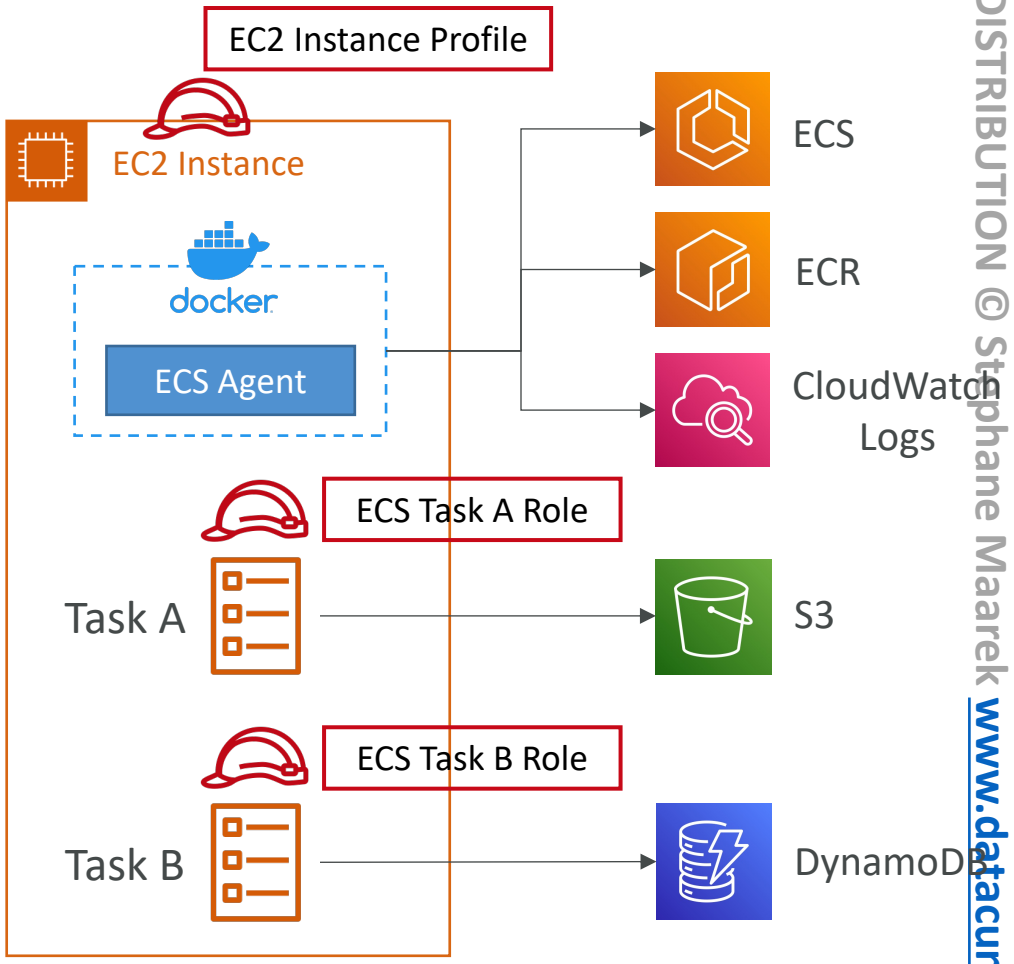
- Launch Docker containers on AWS
- **You do not provision the infrastructure (no EC2 instances to manage)**
- **It's all Serverless!**
- You just create task definitions
- AWS just runs ECS Tasks for you based on the CPU / RAM you need
- To scale, just increase the number of tasks. Simple - no more EC2 instances



Amazon ECS – IAM Roles for ECS

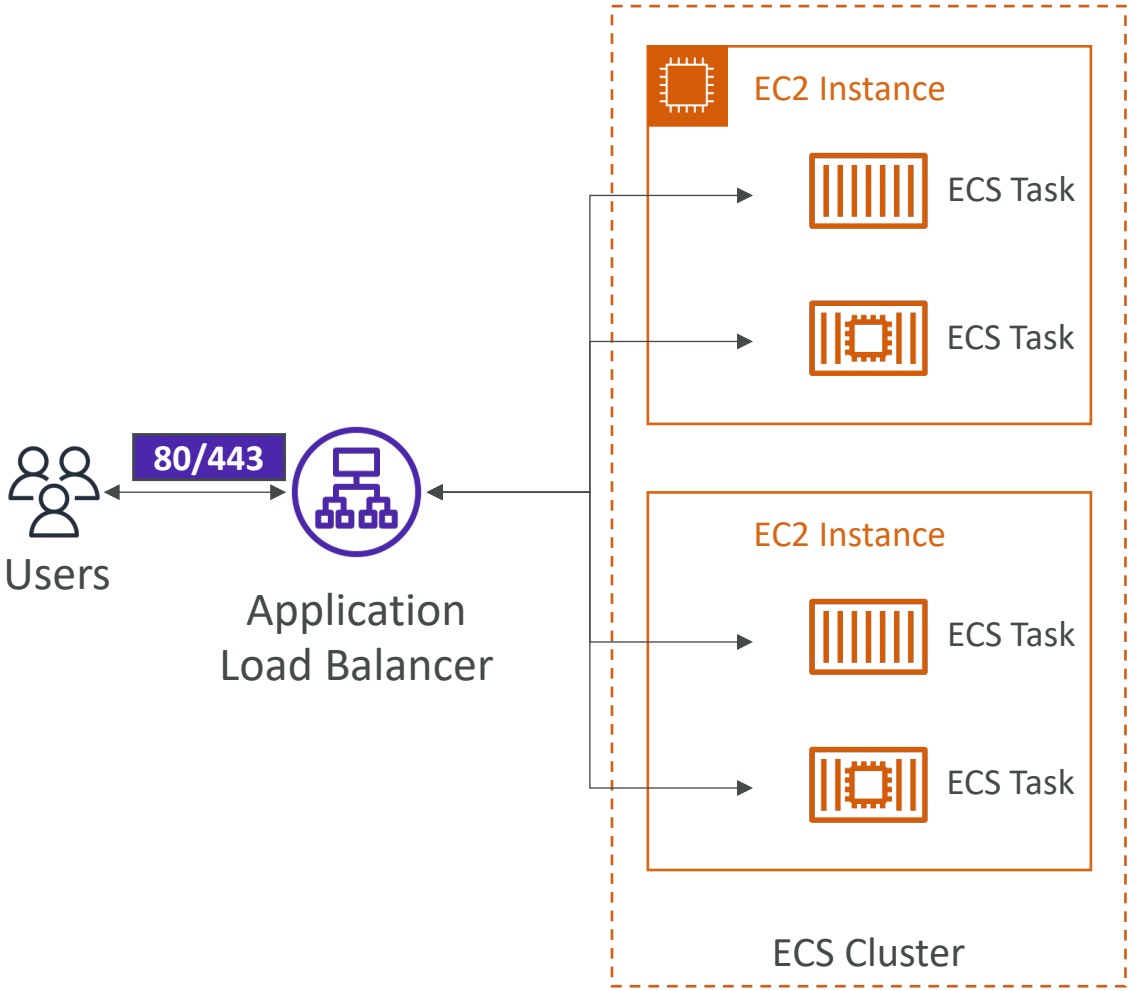
- EC2 Instance Profile (EC2 Launch Type only):
  - Used by the ECS agent
  - Makes API calls to ECS service
  - Send container logs to CloudWatch Logs
  - Pull Docker image from ECR
  - Reference sensitive data in Secrets Manager or SSM Parameter Store

- ECS Task Role:
  - Allows each task to have a specific role
  - Use different roles for the different ECS Services you run
  - Task Role is defined in the task definition



Amazon ECS – Load Balancer Integrations

- Application Load Balancer supported and works for most use cases
- Network Load Balancer recommended only for high throughput / high performance use cases, or to pair it with AWS Private Link
- Classic Load Balancer supported but not recommended (no advanced features – no Fargate)

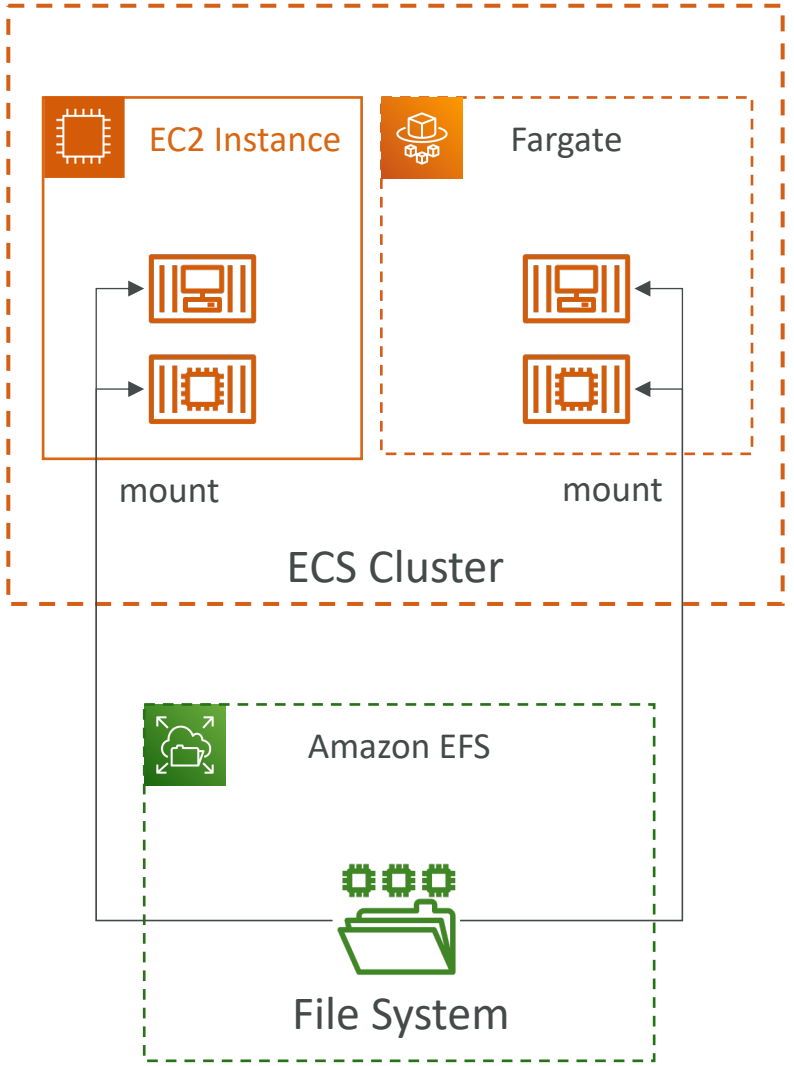


Amazon ECS – Data Volumes (EFS)

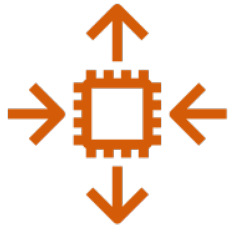
- Mount EFS file systems onto ECS tasks
- Works for both **EC2** and **Fargate** launch types
- Tasks running in any AZ will share the same data in the EFS file system
- **Fargate + EFS = Serverless**

- Use cases: persistent multi-AZ shared storage for your containers

- Note:
  - Amazon S3 cannot be mounted as a file system



ECS Service Auto Scaling



- Automatically increase/decrease the desired number of ECS tasks

- Amazon ECS Auto Scaling uses **AWS Application Auto Scaling**
  - ECS Service Average CPU Utilization
  - ECS Service Average Memory Utilization - Scale on RAM
  - ALB Request Count Per Target – metric coming from the ALB

- **Target Tracking** – scale based on target value for a specific CloudWatch metric
- **Step Scaling** – scale based on a specified CloudWatch Alarm
- **Scheduled Scaling** – scale based on a specified date/time (predictable changes)

- ECS Service Auto Scaling (task level) ≠ EC2 Auto Scaling (EC2 instance level)
- Fargate Auto Scaling is much easier to setup (because **Serverless**)

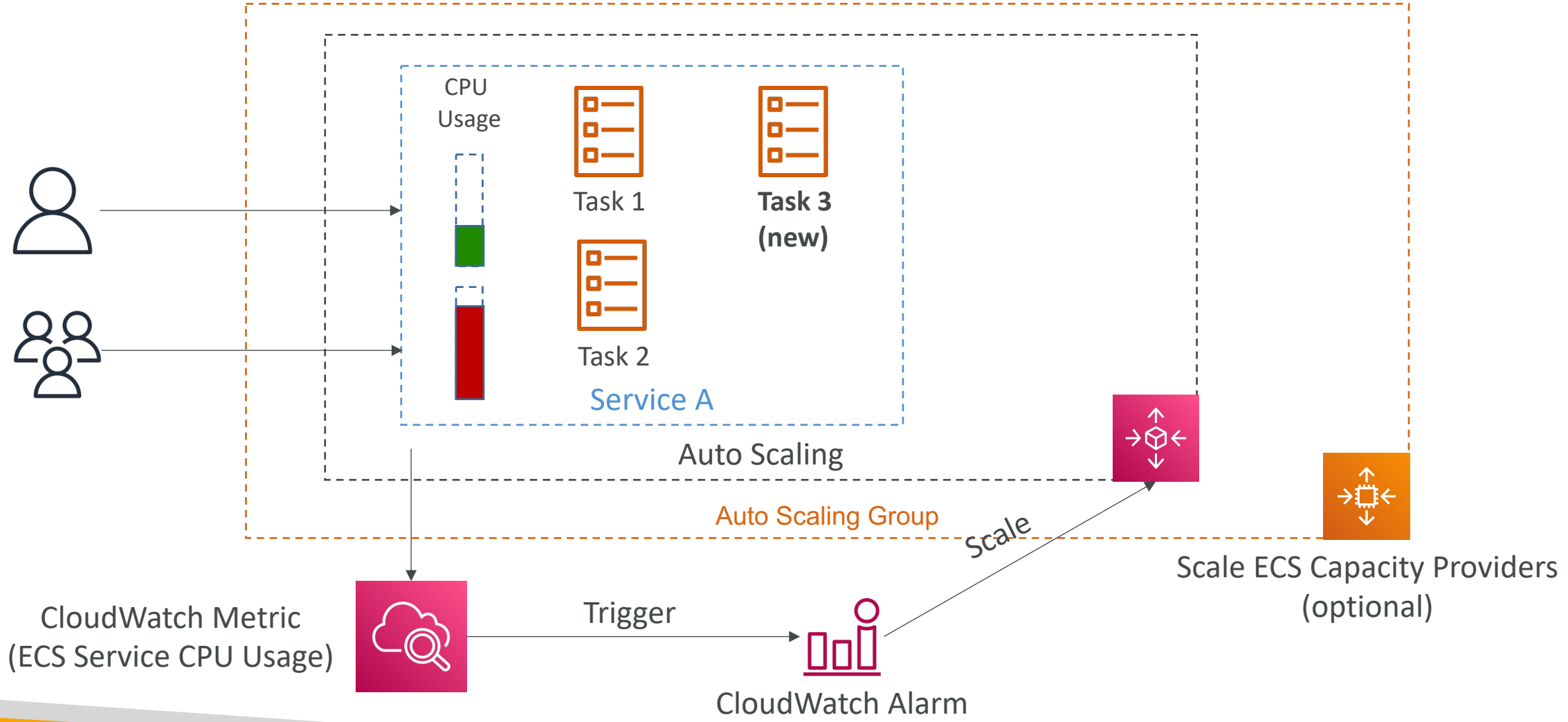
EC2 Launch Type – Auto Scaling EC2 Instances

- Accommodate ECS Service Scaling by adding underlying EC2 Instances

- Auto Scaling Group Scaling
  - Scale your ASG based on CPU Utilization
  - Add EC2 instances over time

- ECS Cluster Capacity Provider
  - Used to automatically provision and scale the infrastructure for your ECS Tasks
  - Capacity Provider paired with an Auto Scaling Group
  - Add EC2 Instances when you're missing capacity (CPU, RAM…)

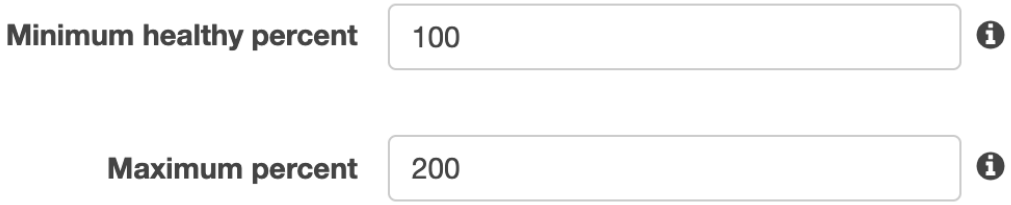
ECS Scaling – Service CPU Usage Example

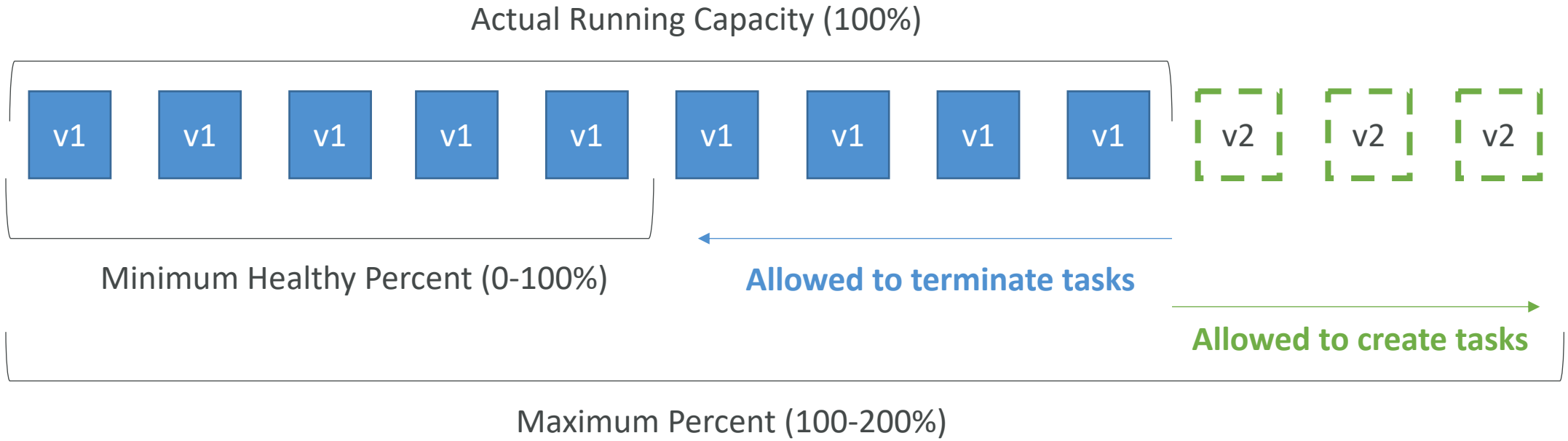


ECS Rolling Updates

- When updating from v1 to v2, we can control how many tasks can be started and stopped, and in which order

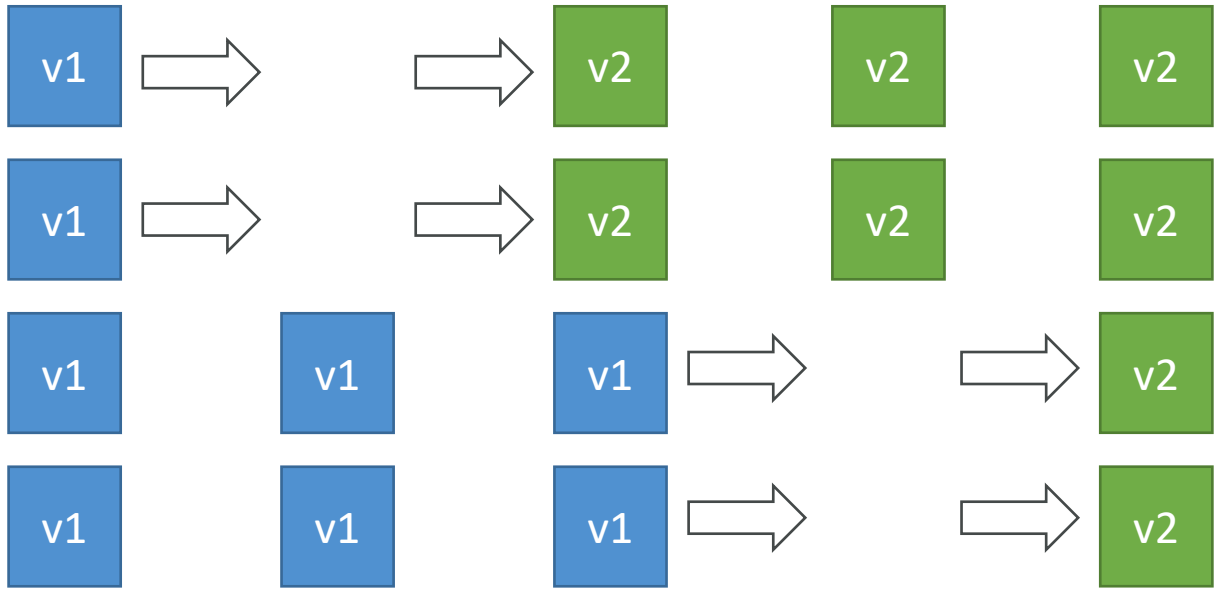
ECS Service update screen





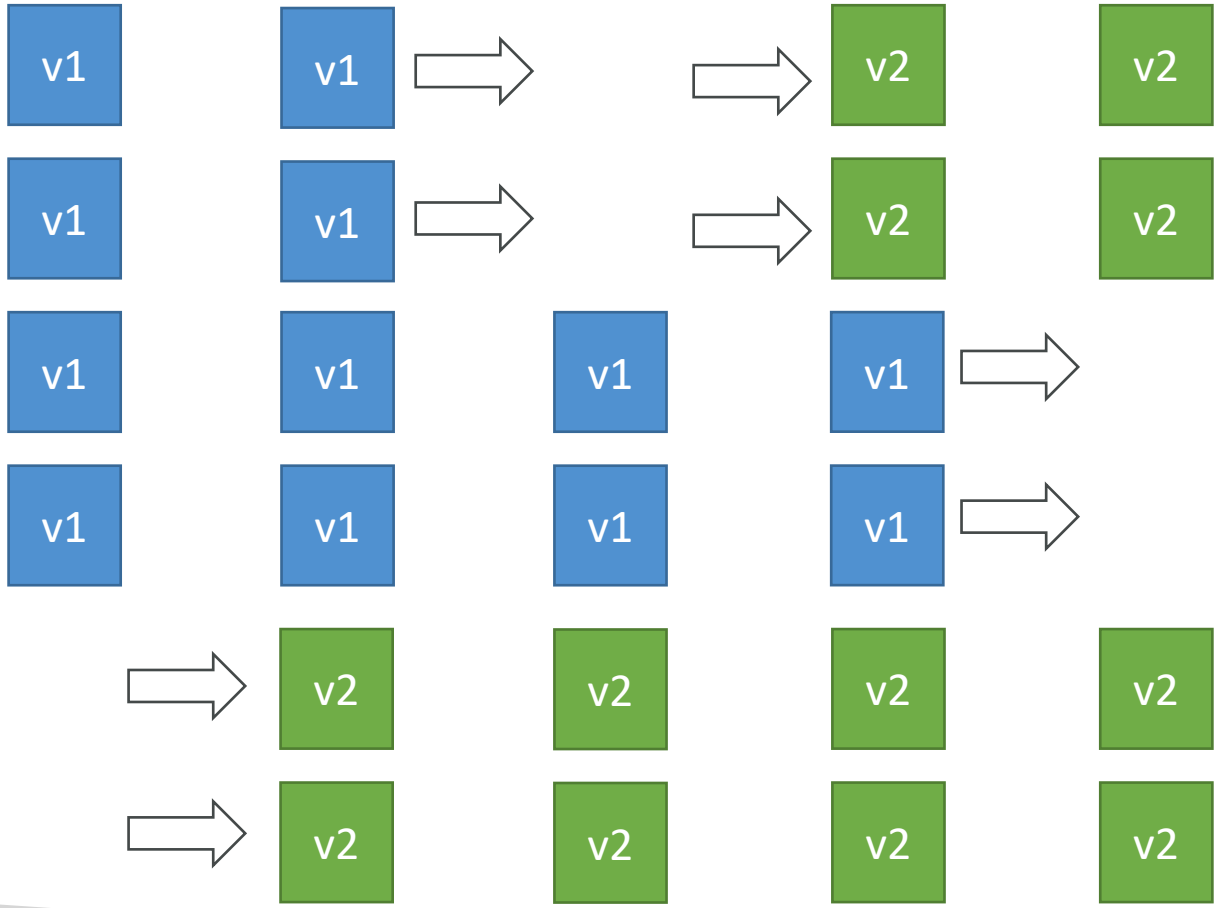
ECS Rolling Update – **Min 50%**, Max 100%

- Starting number of tasks: 4

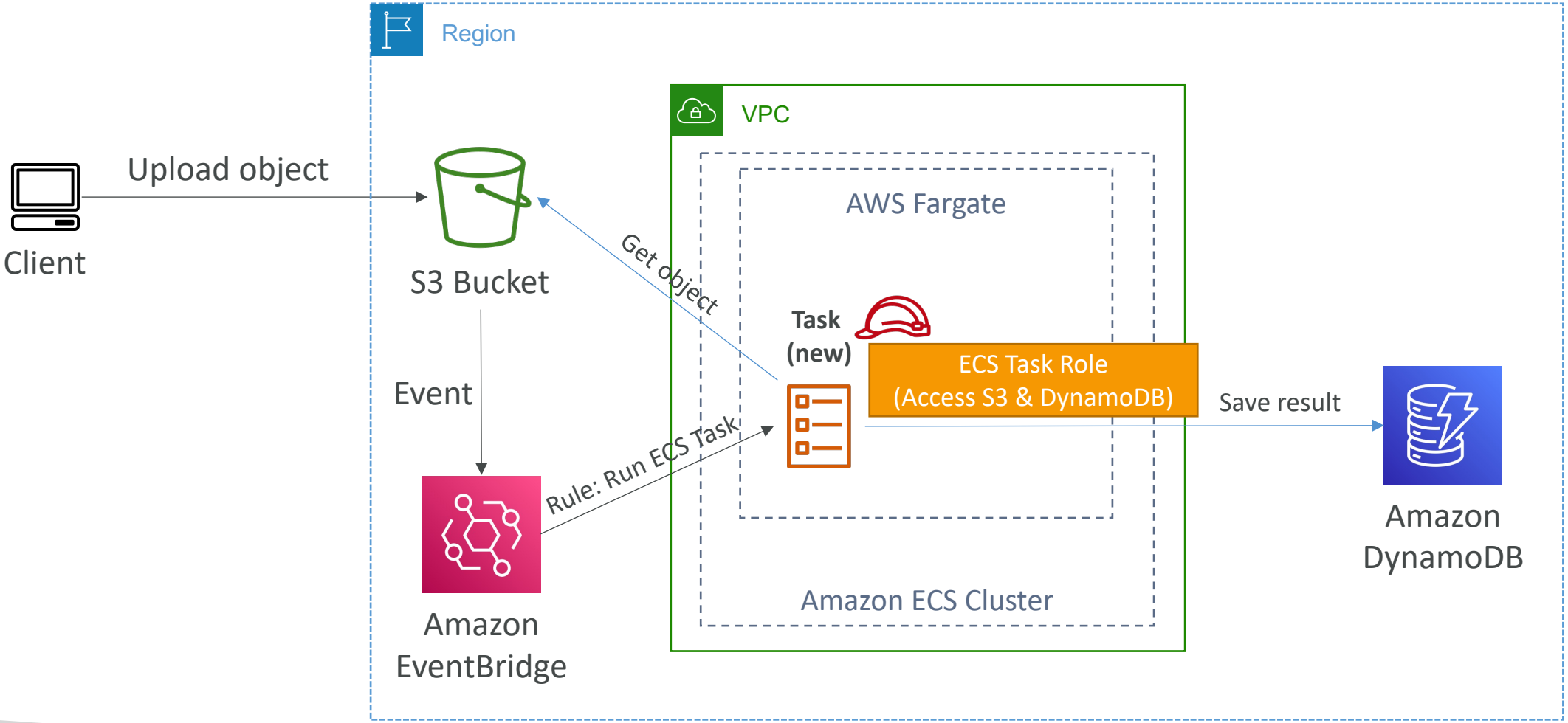


ECS Rolling Update – Min 100%, Max 150%

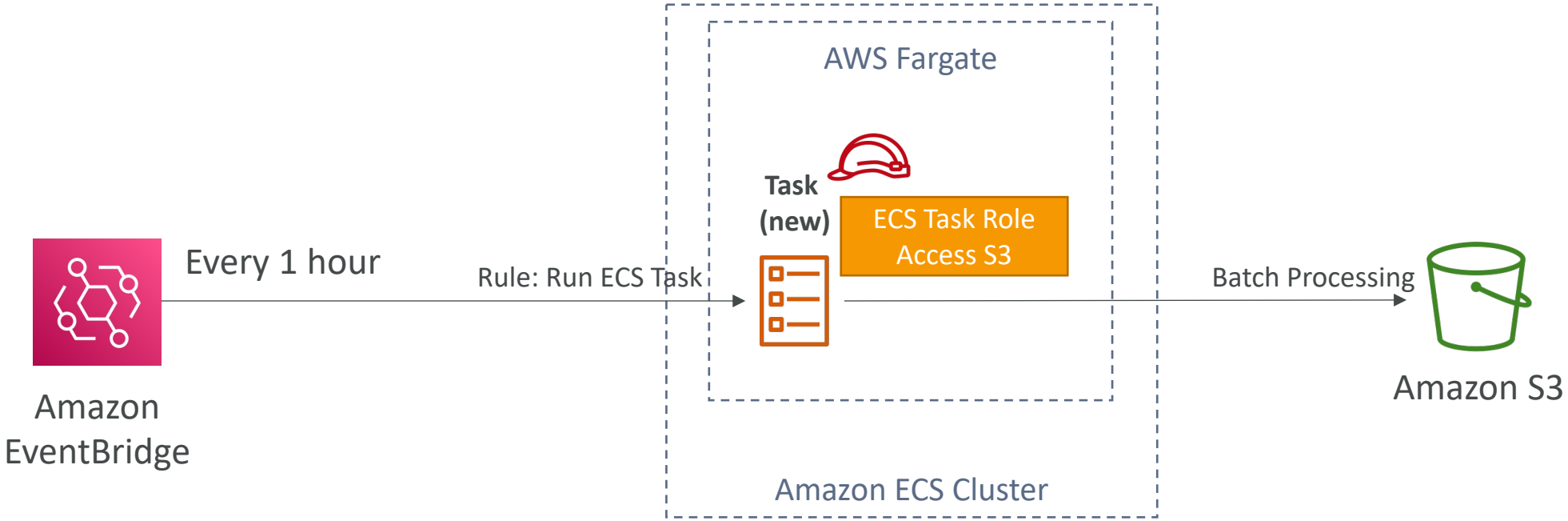
- Starting number of tasks: 4



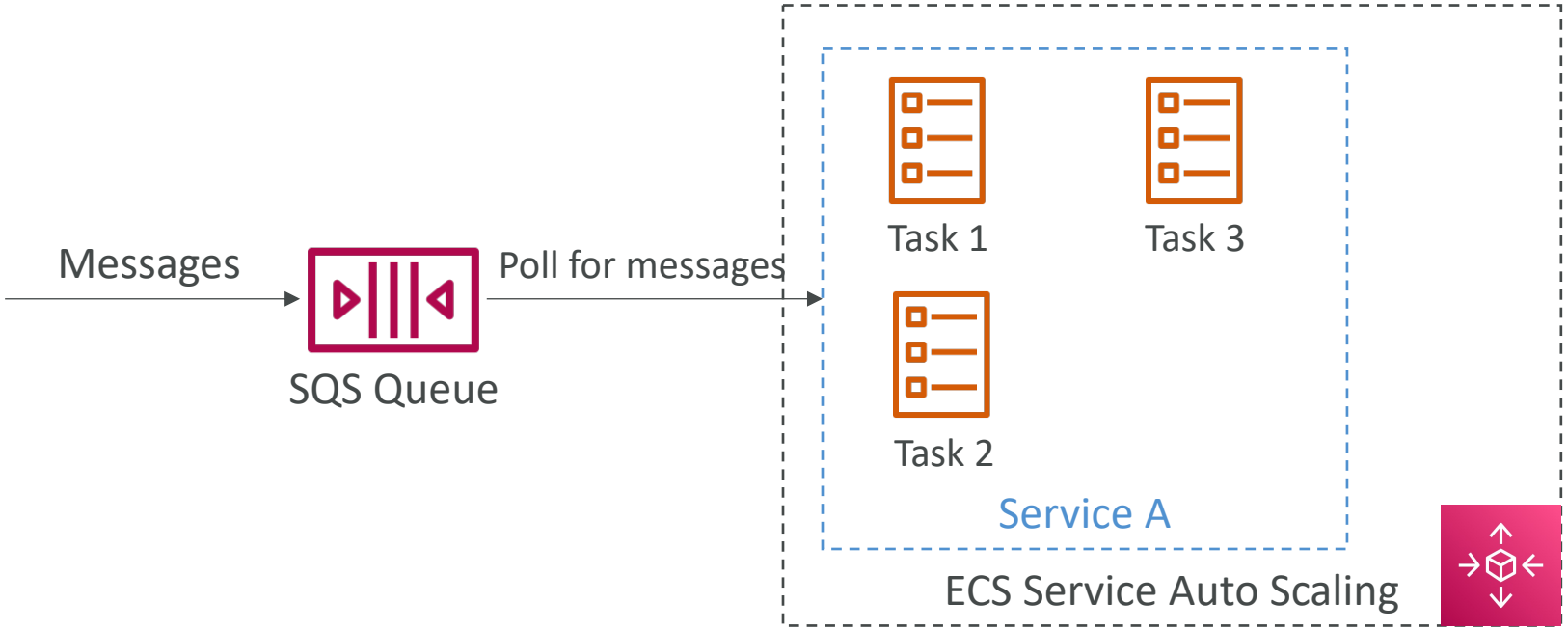
ECS tasks invoked by Event Bridge



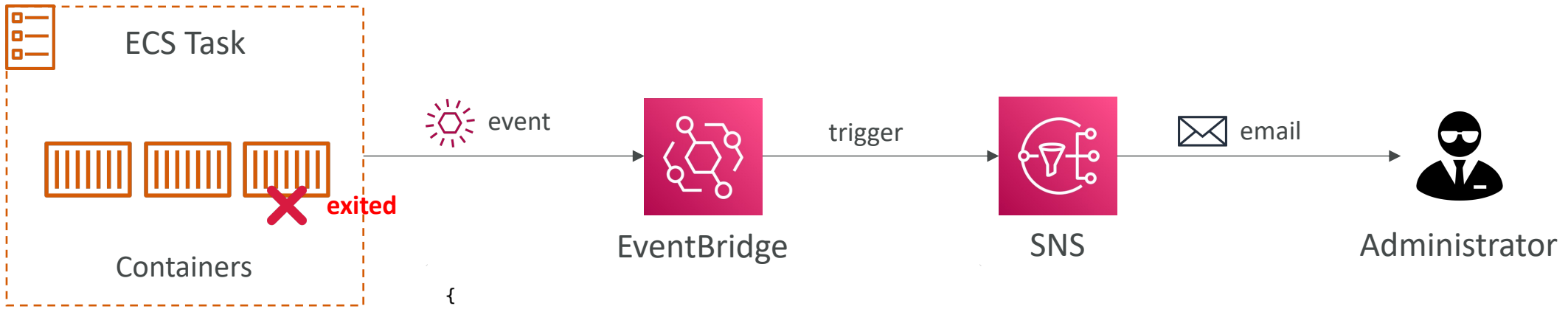
ECS tasks invoked by Event Bridge Schedule



ECS – SQS Queue Example



ECS – Intercept Stopped Tasks using EventBridge

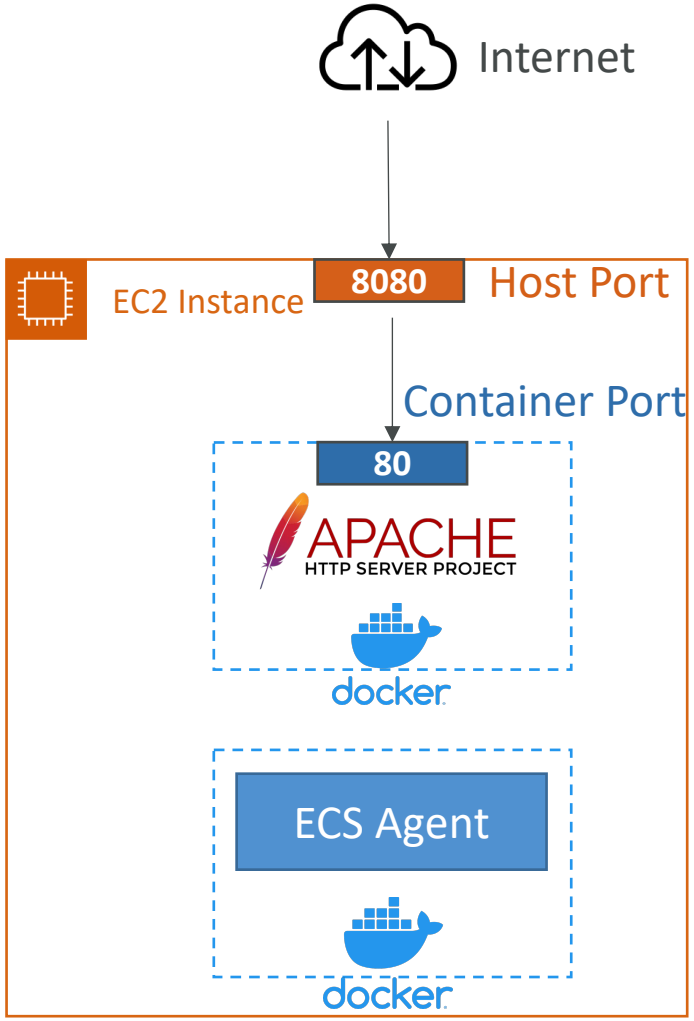



```
{
    "source": [
        "aws.ecs"
    ],
    "detail-type": [
        "ECS Task State Change"
    ],
    "detail": {
        "lastStatus": [
            "STOPPED"
        ],
        "stoppedReason": [
            "Essential container in task exited"
        ]
    }
}
```

Event Pattern

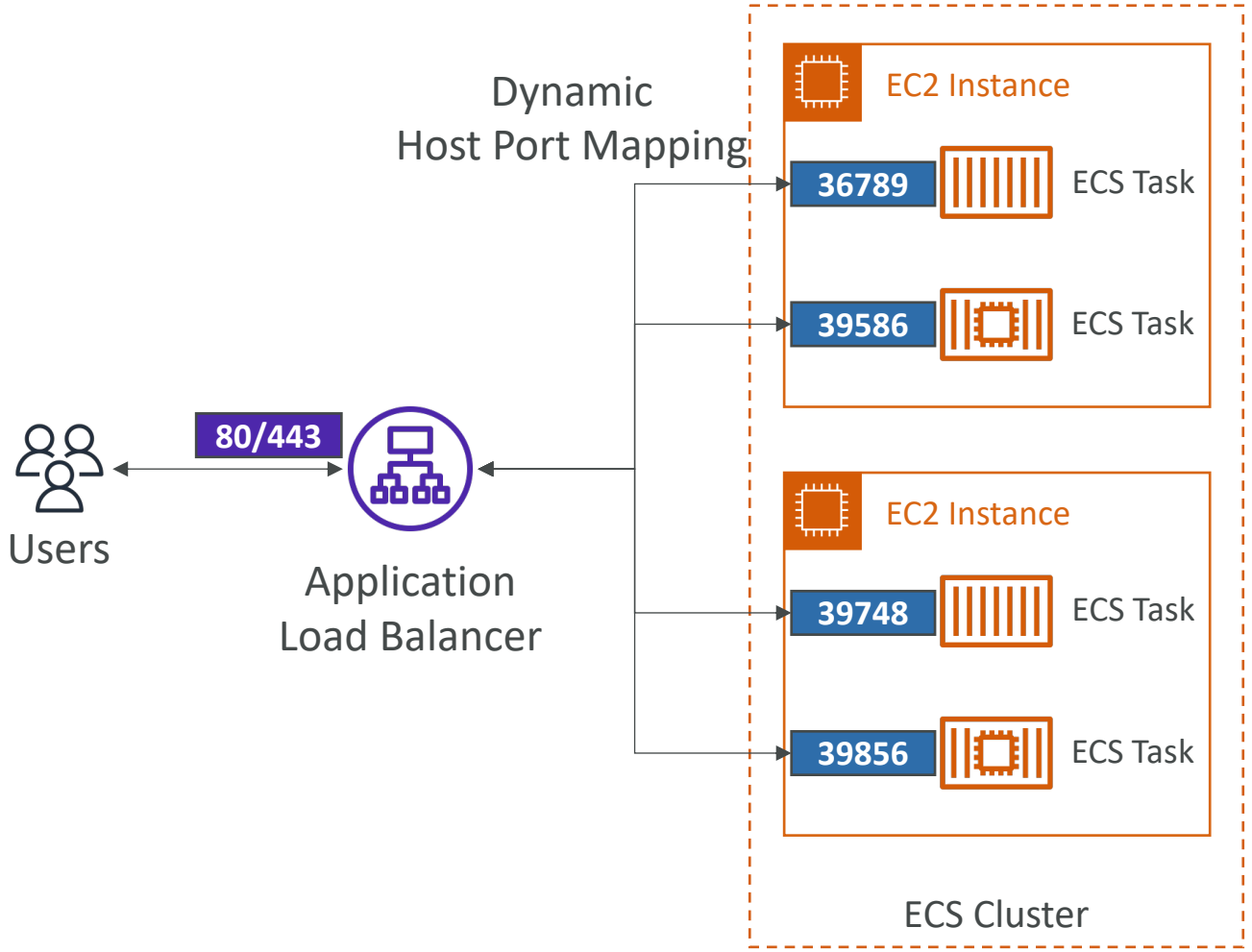
Amazon ECS – Task Definitions

- Task definitions are metadata in **JSON form** to tell ECS how to run a Docker container
- It contains crucial information, such as:
  - Image Name
  - Port Binding for Container and Host
  - Memory and CPU required
  - Environment variables
  - Networking information
  - IAM Role
  - Logging configuration (ex CloudWatch)
- Can define up to 10 containers in a Task Definition



Amazon ECS – Load Balancing (EC2 Launch Type)

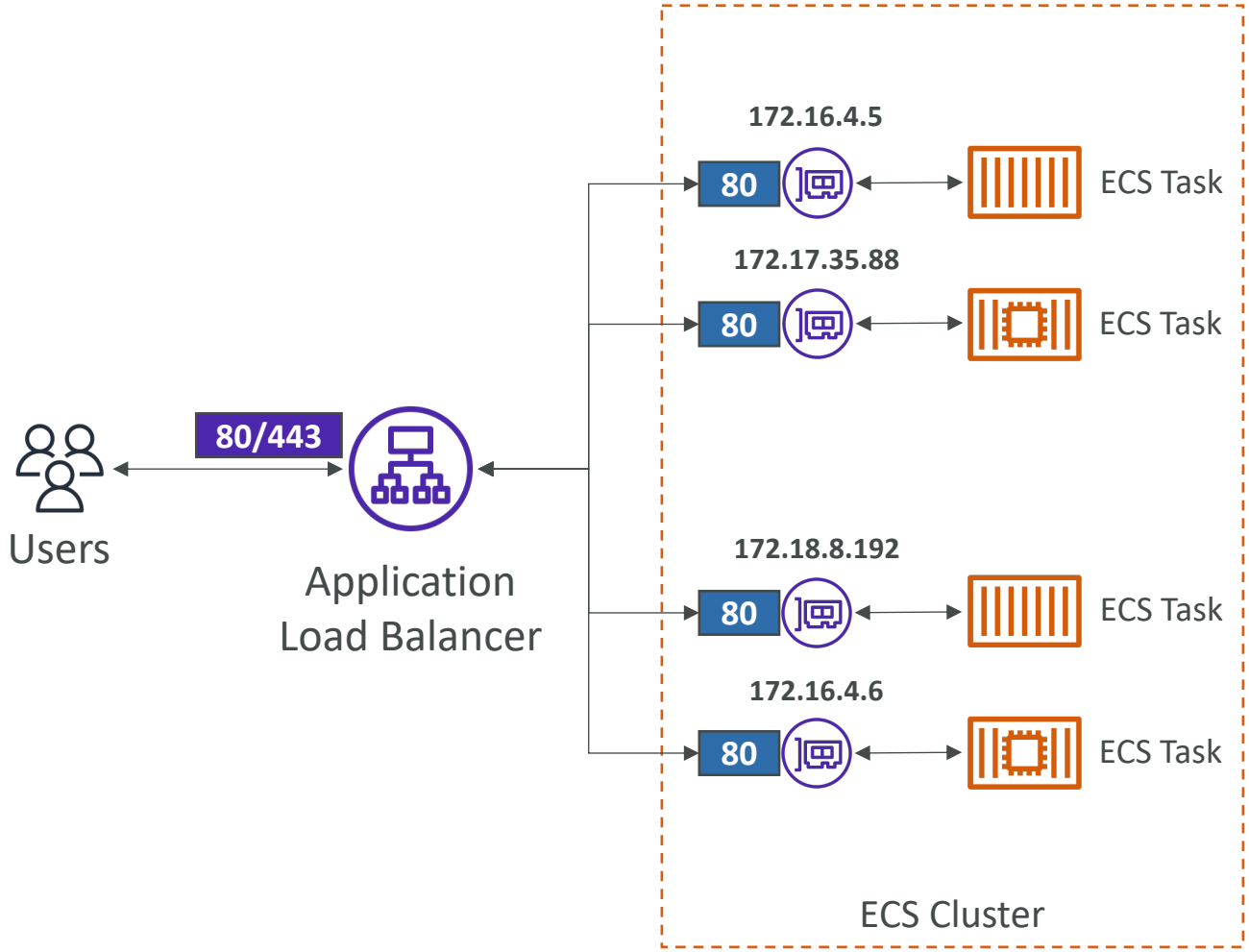
- We get a Dynamic Host Port Mapping if you define only the container port in the task definition
- The ALB finds the right port on your EC2 Instances
- You must allow on the EC2 instance's Security Group any port from the ALB's Security Group



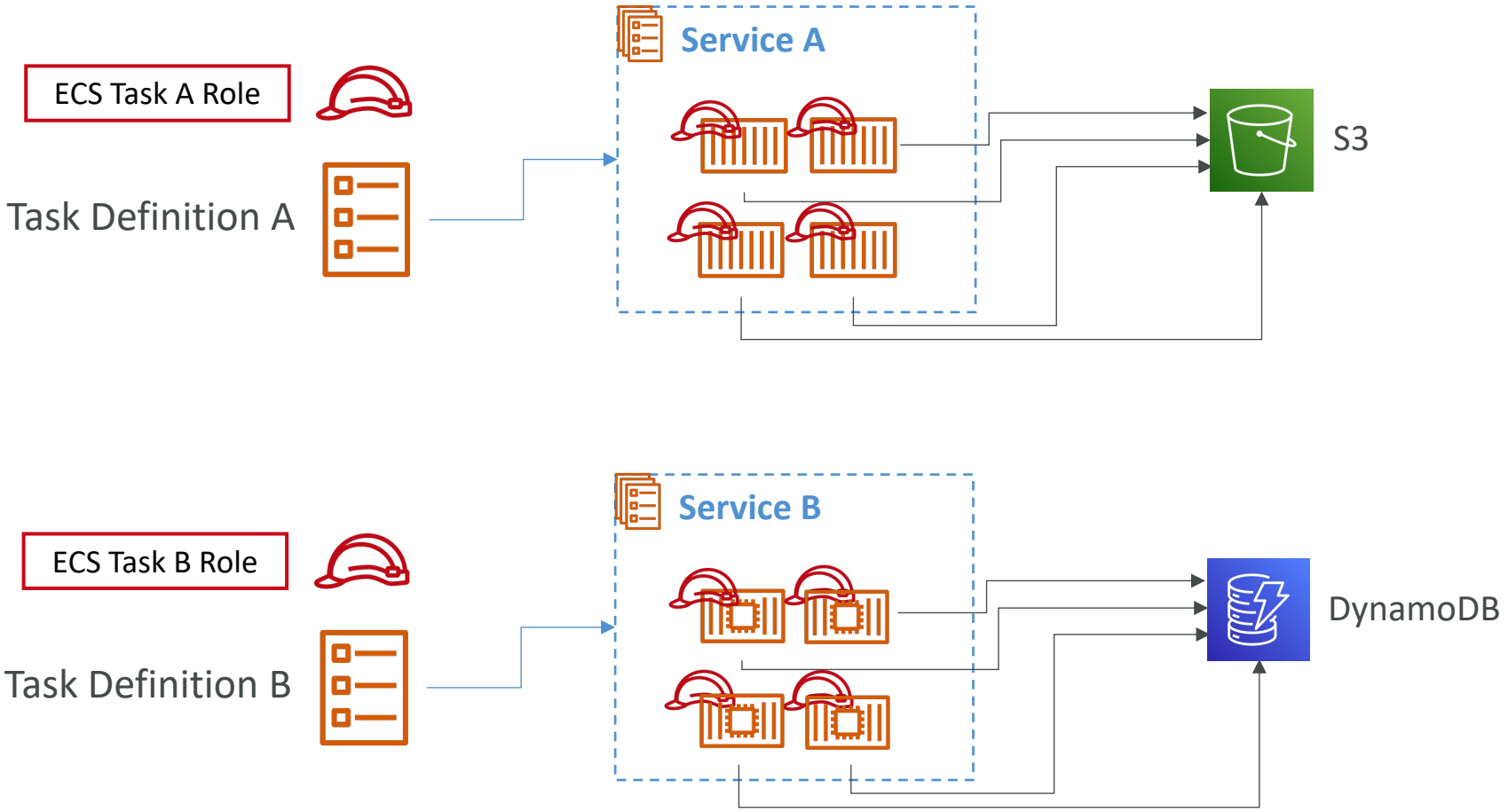
Amazon ECS – Load Balancing (Fargate)

- Each task has a **unique private IP**
- **Only define the container port** (host port is not applicable)

- Example
  - ECS ENI Security Group
    - Allow port 80 from the ALB
  - ALB Security Group
    - Allow port 80/443 from web

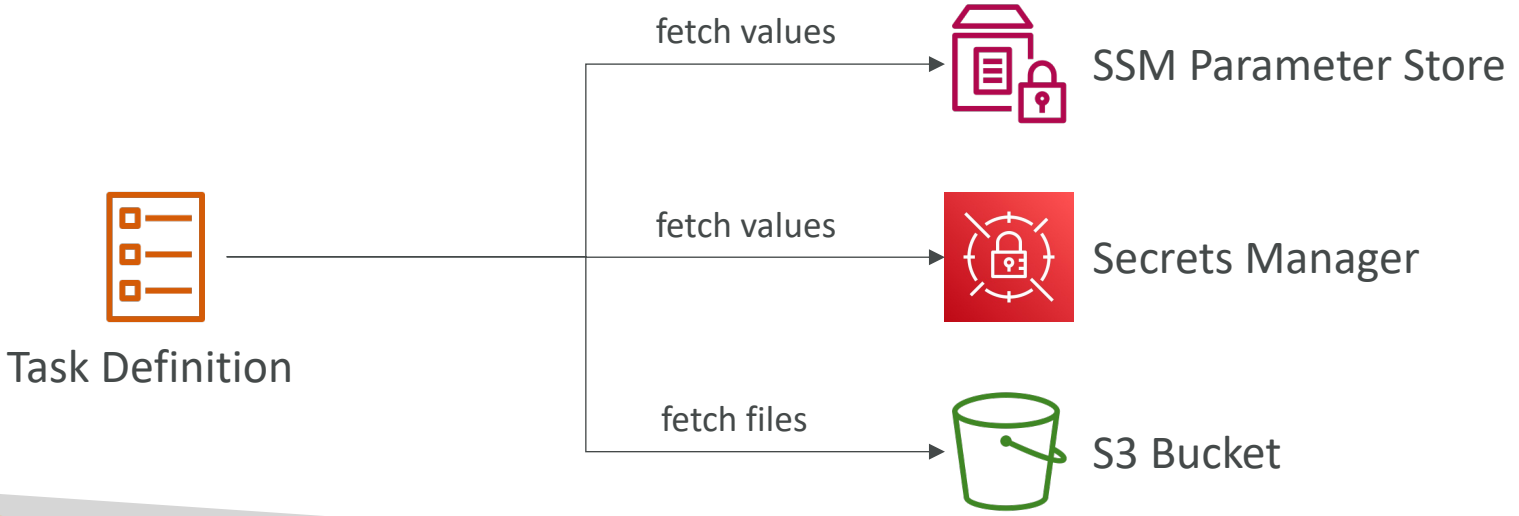


Amazon ECS
One IAM Role per Task Definition



Amazon ECS – Environment Variables

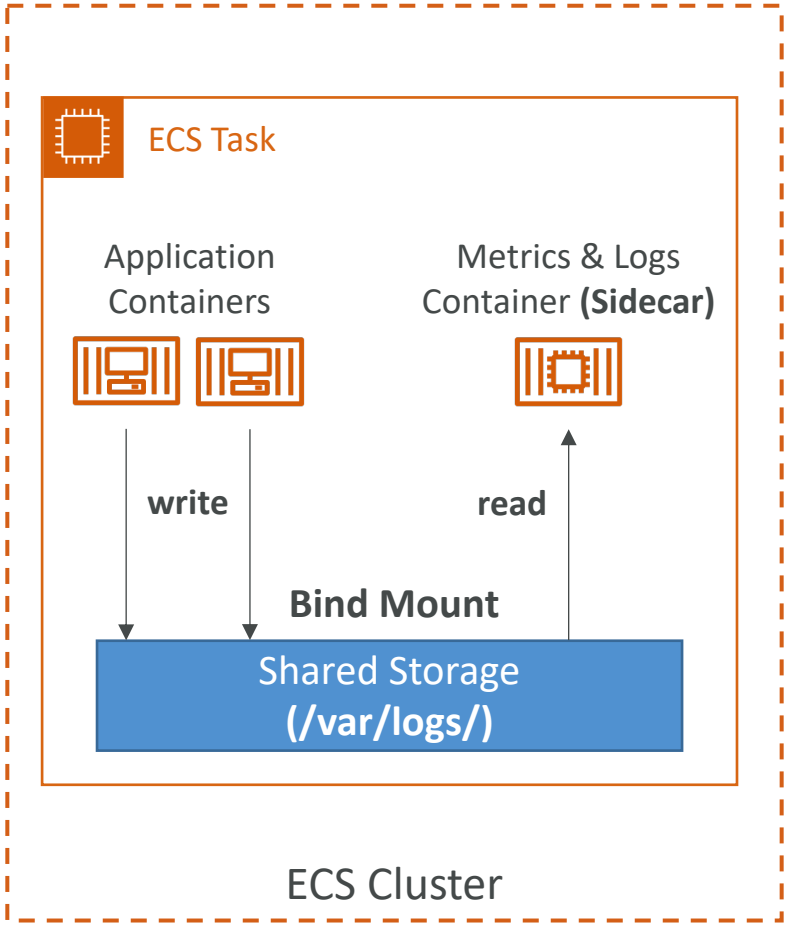
- Environment Variable
  - **Hardcoded** – e.g., URLs
  - **SSM Parameter Store** – sensitive variables (e.g., API keys, shared configs)
  - **Secrets Manager** – sensitive variables (e.g., DB passwords)
- Environment Files (bulk) – Amazon S3



Amazon ECS – Data Volumes (Bind Mounts)

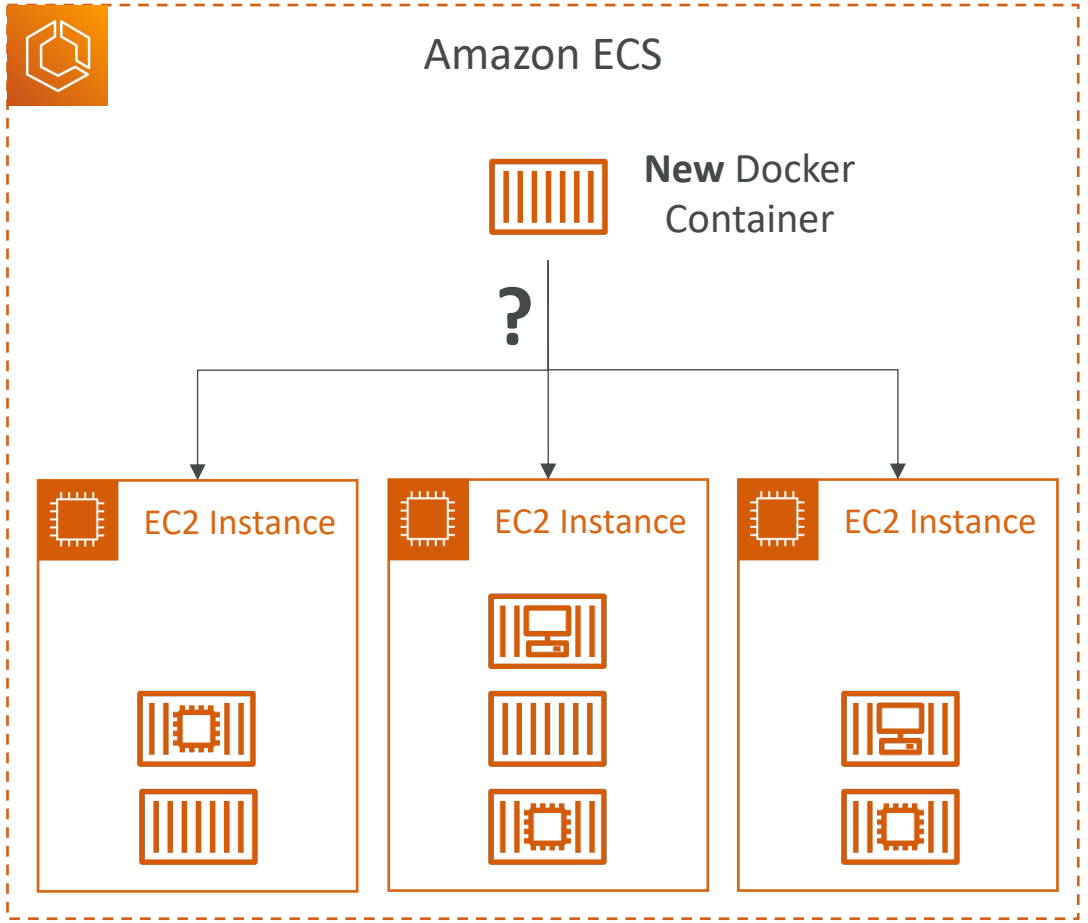
- Share data between multiple containers in the same Task Definition
- Works for both **EC2** and **Fargate** tasks
- **EC2 Tasks** – using EC2 instance storage
  - Data are tied to the lifecycle of the EC2 instance
- **Fargate Tasks** – using ephemeral storage
  - Data are tied to the container(s) using them
  - 20 GiB – 200 GiB (default 20 GiB)

- Use cases:
  - Share ephemeral data between multiple containers
  - "Sidecar" container pattern, where the "sidecar" container used to send metrics/logs to other destinations (separation of conerns)



Amazon ECS – Task Placement

- When an ECS task is started with EC2 Launch Type, ECS must determine where to place it, with the constraints of **CPU** and **memory (RAM)**
- Similarly, when a service scales in, ECS needs to determine which task to terminate
- You can define:
  - Task Placement Strategy
  - Task Placement Constraints
- Note: only for ECS Tasks with EC2 Launch Type **(Fargate not supported)**



Amazon ECS – Task Placement Process

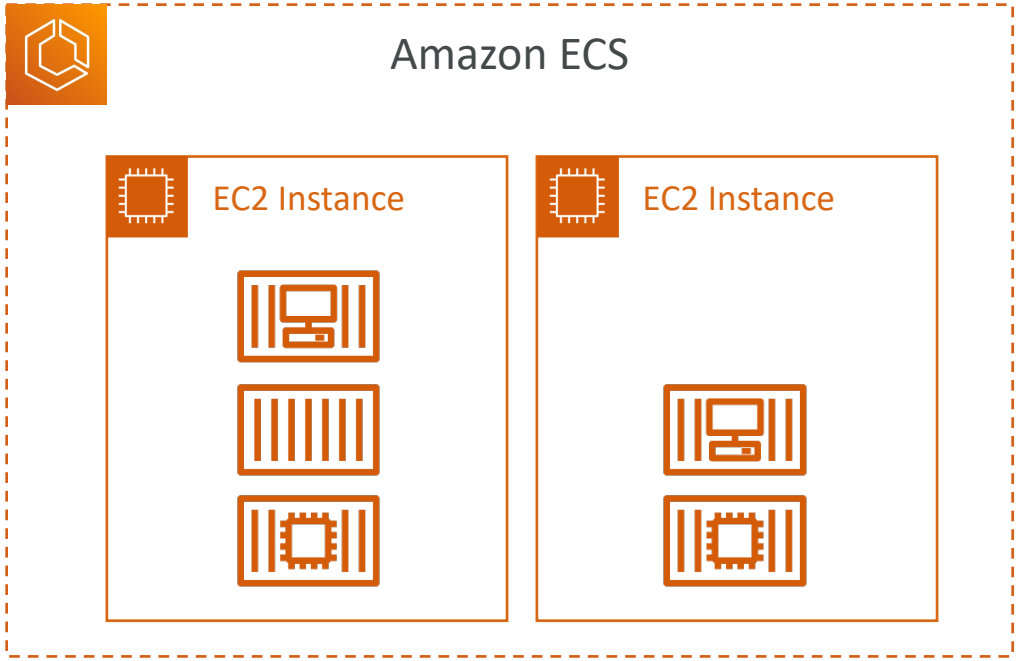
- Task Placement Strategies are a **best effort**

- When Amazon ECS places a task, it uses the following process to select the appropriate EC2 Container instance:
  1. Identify which instances that satisfy the **CPU, memory, and port** requirements
  2. Identify which instances that satisfy the **Task Placement Constraints**
  3. Identify which instances that satisfy the **Task Placement Strategies**
  4. Select the instances

Amazon ECS – Task Placement Strategies

- Binpack
  - Tasks are placed on the least available amount of CPU and Memory
  - Minimizes the number of EC2 instances in use (cost savings)

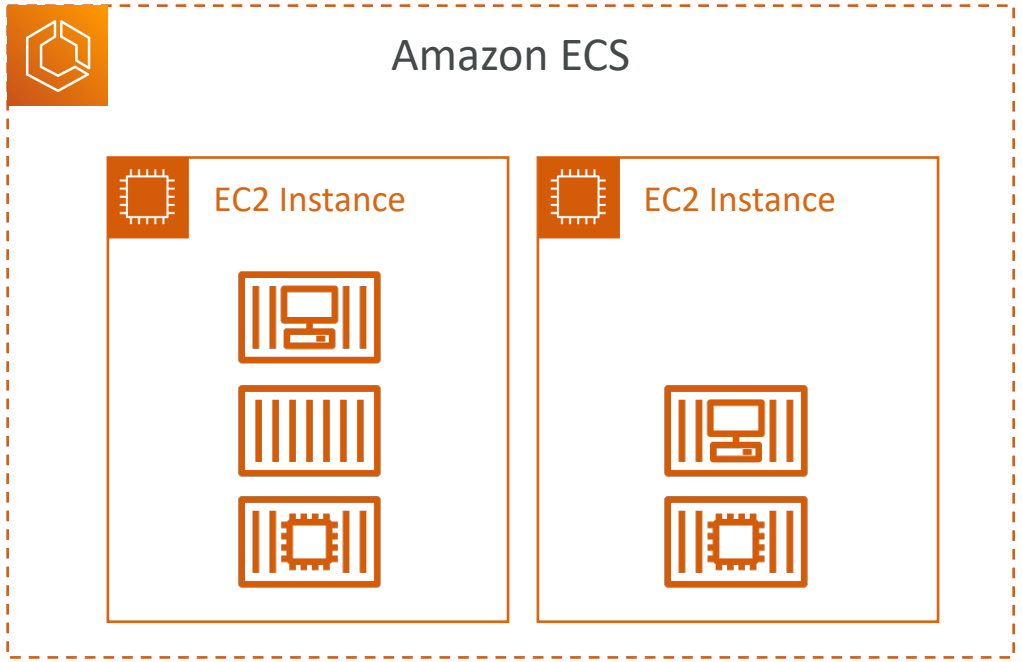
```
"placementStrategy": [
    {
        "type": "binpack",
        "field": "memory"
    }
]
```



Amazon ECS – Task Placement Strategies

- Random
  - Tasks are placed randomly

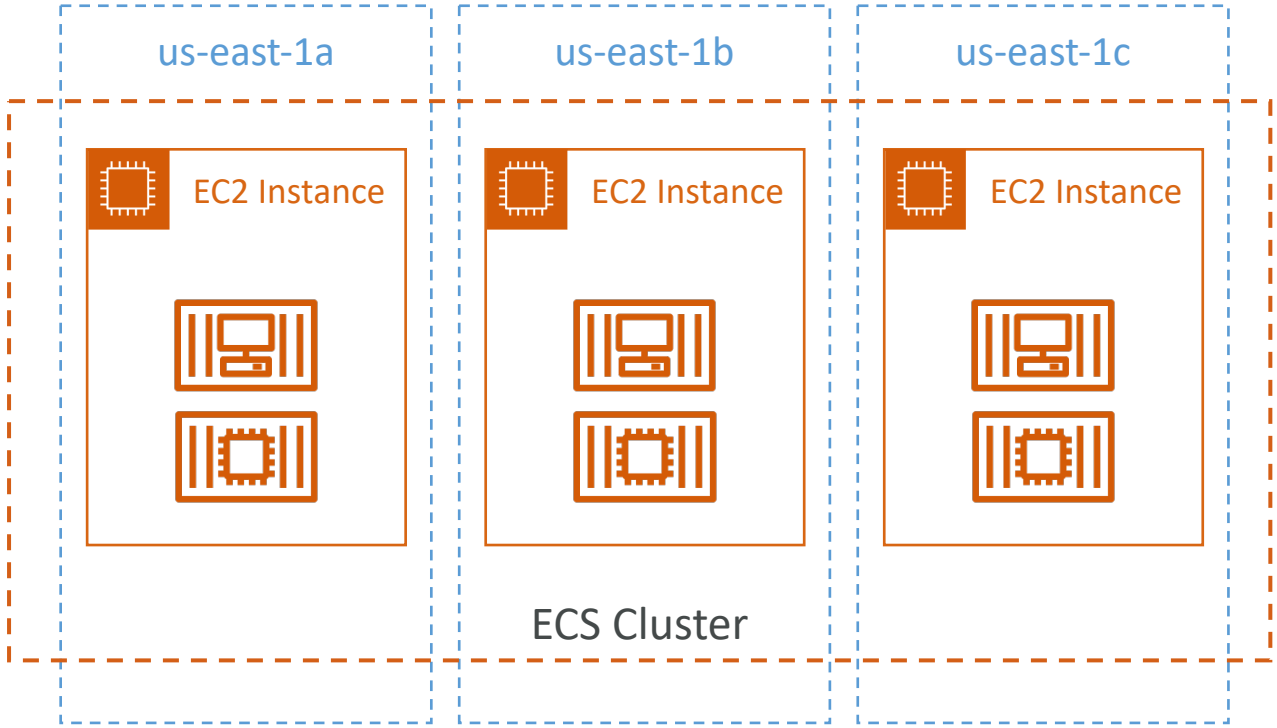
```
"placementStrategy": [
    {
        "type": "random"
    }
]
```



Amazon ECS – Task Placement Strategies

- Spread
  - Tasks are placed evenly based on the specified value
  - Example: **instanceId**, **attribute:ecs.availability-zone**, …

```
"placementStrategy": [
    {
        "type": "spread",
        "field": "attribute:ecs.availability-zone"
    }
]
```



Amazon ECS – Task Placement Strategies

- You can mix them together

```
"placementStrategy": [
    {
        "type": "spread",
        "field": "attribute:ecs.availability-zone"
    },
    {
        "type": "spread",
        "field": "instanceId"
    }
]
```

```
"placementStrategy": [
    {
        "type": "spread",
        "field": "attribute:ecs.availability-zone"
    },
    {
        "type": "binpack",
        "field": "memory"
    }
]
```

Amazon ECS – Task Placement Constraints

- distinctInstance
  - Tasks are placed on a different EC2 instance

```
"placementConstraints": [
    {
        "type": "distinctInstance"
    }
]
```

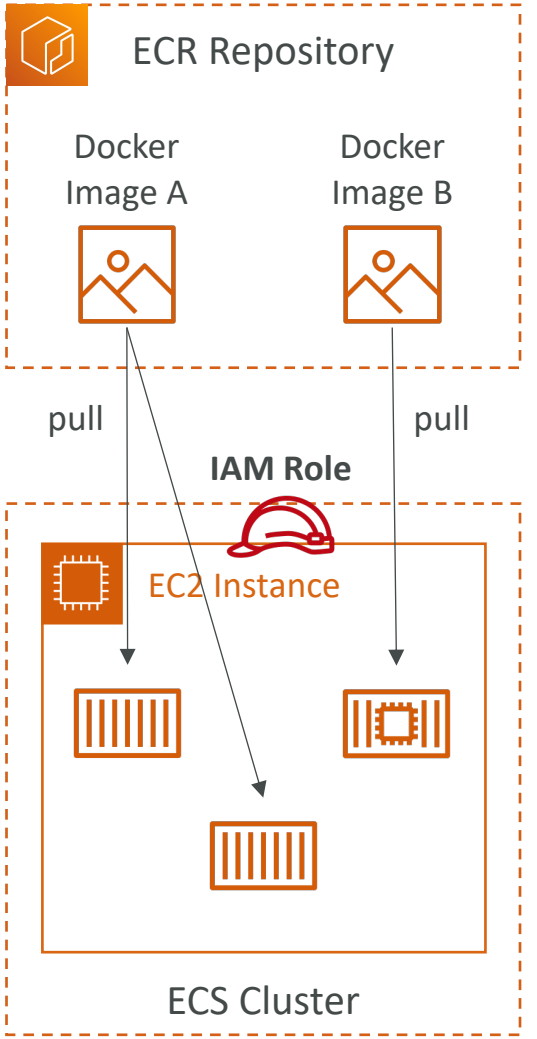
- memberOf
  - Tasks are placed on EC2 instances that satisfy a specified expression
  - Uses the **Cluster Query Language** (advanced)

```
"placementConstraints": [
    {
        "type": "memberOf",
        "expression": "attribute:ecs.instance-type =~ t2.*"
    }
]
```

```
"placementConstraints": [
    {
        "type": "memberOf",
        "expression": "attribute:ecs.availability-zone in
 [eu-west-2a, eu-west-2b]"
    }
]
```

Amazon ECR

- ECR = Elastic Container Registry
- Store and manage Docker images on AWS
- **Private** and **Public** repository (**Amazon ECR Public Gallery** https://gallery.ecr.aws)
- Fully integrated with ECS, backed by Amazon S3
- Access is controlled through IAM (permission errors => policy)
- Supports image vulnerability scanning, versioning, image tags, image lifecycle, …



Amazon ECR – Using AWS CLI

- Login Command
  - AWS CLI v2

```
aws ecr get-login-password --region region | docker login --username AWS
      --password-stdin aws_account_id.dkr.ecr.region.amazonaws.com
```

- Docker Commands
  - Push

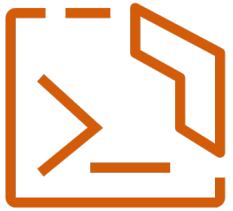
```
docker push aws_account_id.dkr.ecr.region.amazonaws.com/demo:latest
```

  - Pull

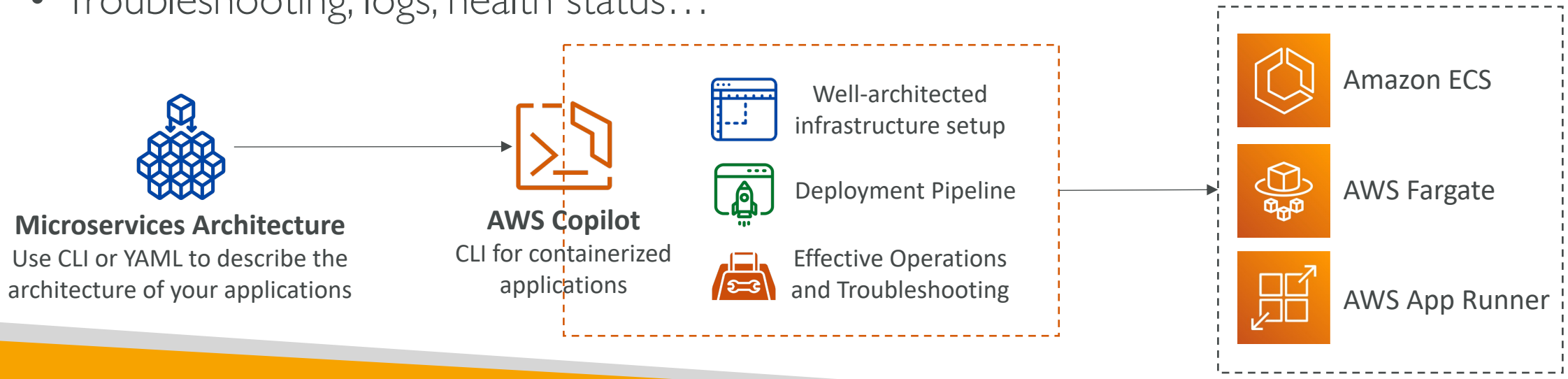
```
docker pull aws_account_id.dkr.ecr.region.amazonaws.com/demo:latest
```

  - In case an EC2 instance (or you) can't pull a Docker image, check IAM permissions

AWS Copilot



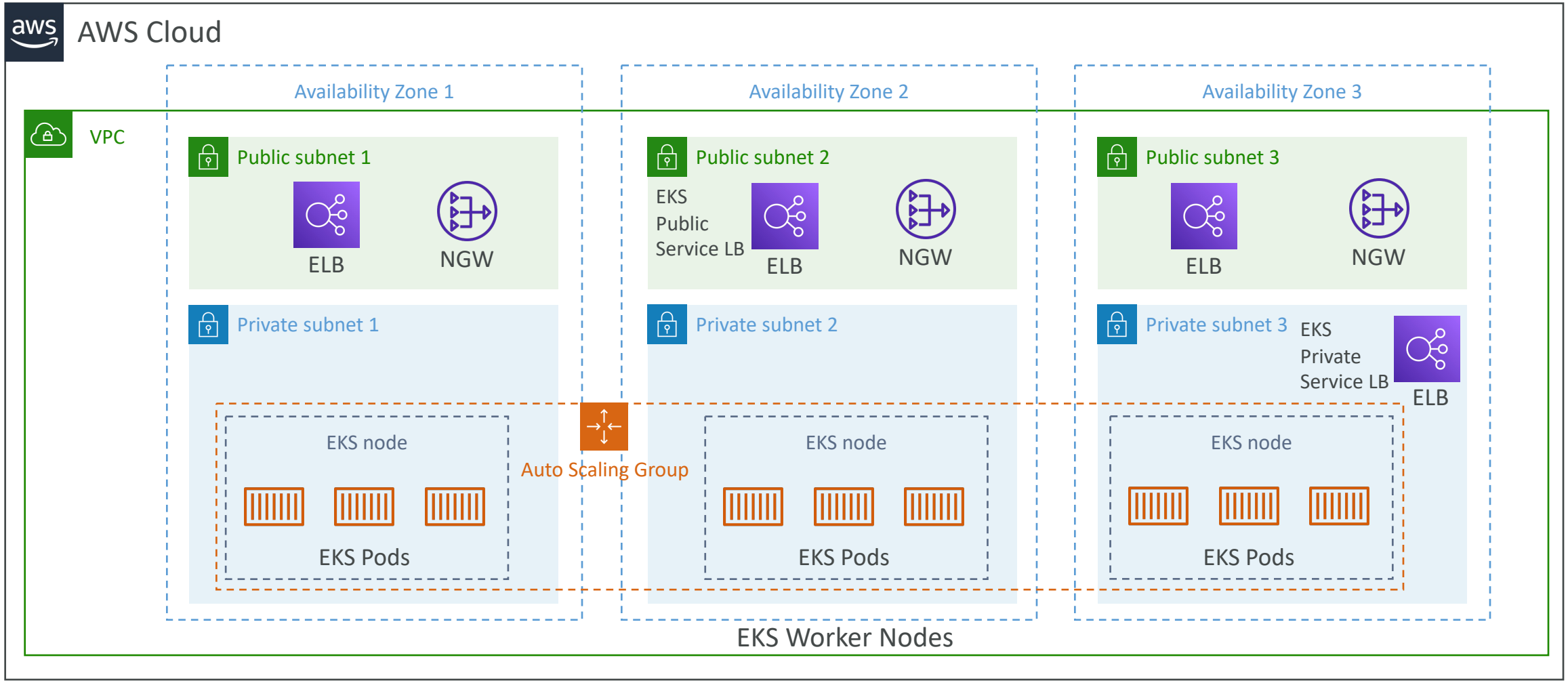
- CLI tool to build, release, and operate production-ready containerized apps
- Run your apps on **AppRunner**, **ECS**, and **Fargate**
- Helps you focus on building apps rather than setting up infrastructure
- Provisions all required infrastructure for containerized apps (ECS, VPC, ELB, ECR…)
- Automated deployments with one command using CodePipeline
- Deploy to multiple environments
- Troubleshooting, logs, health status…



Amazon EKS Overview

- Amazon EKS = Amazon Elastic **Kubernetes** Service
- It is a way to launch **managed Kubernetes clusters on AWS**
- Kubernetes is an **open-source system** for automatic deployment, scaling and management of containerized (usually Docker) application
- It's an alternative to ECS, similar goal but different API
- EKS supports **EC2** if you want to deploy worker nodes or **Fargate** to deploy serverless containers
- **Use case:** if your company is already using Kubernetes on-premises or in another cloud, and wants to migrate to AWS using Kubernetes
- **Kubernetes is cloud-agnostic** (can be used in any cloud – Azure, GCP…)
- For multiple regions, deploy one EKS cluster per region
- Collect logs and metrics using **CloudWatch Container Insights**

Amazon EKS - Diagram



Amazon EKS – Node Types

- Managed Node Groups
  - Creates and manages Nodes (EC2 instances) for you
  - Nodes are part of an ASG managed by EKS
  - Supports On-Demand or Spot Instances
- Self-Managed Nodes
  - Nodes created by you and registered to the EKS cluster and managed by an ASG
  - You can use prebuilt AMI - Amazon EKS Optimized AMI
  - Supports On-Demand or Spot Instances
- AWS Fargate
  - No maintenance required; no nodes managed

Amazon EKS – Data Volumes

- Need to specify **StorageClass** manifest on your EKS cluster
- Leverages a **Container Storage Interface (CSI)** compliant driver

- Support for…
- Amazon EBS
- Amazon EFS (works with Fargate)
- Amazon FSx for Lustre
- Amazon FSx for NetApp ONTAP